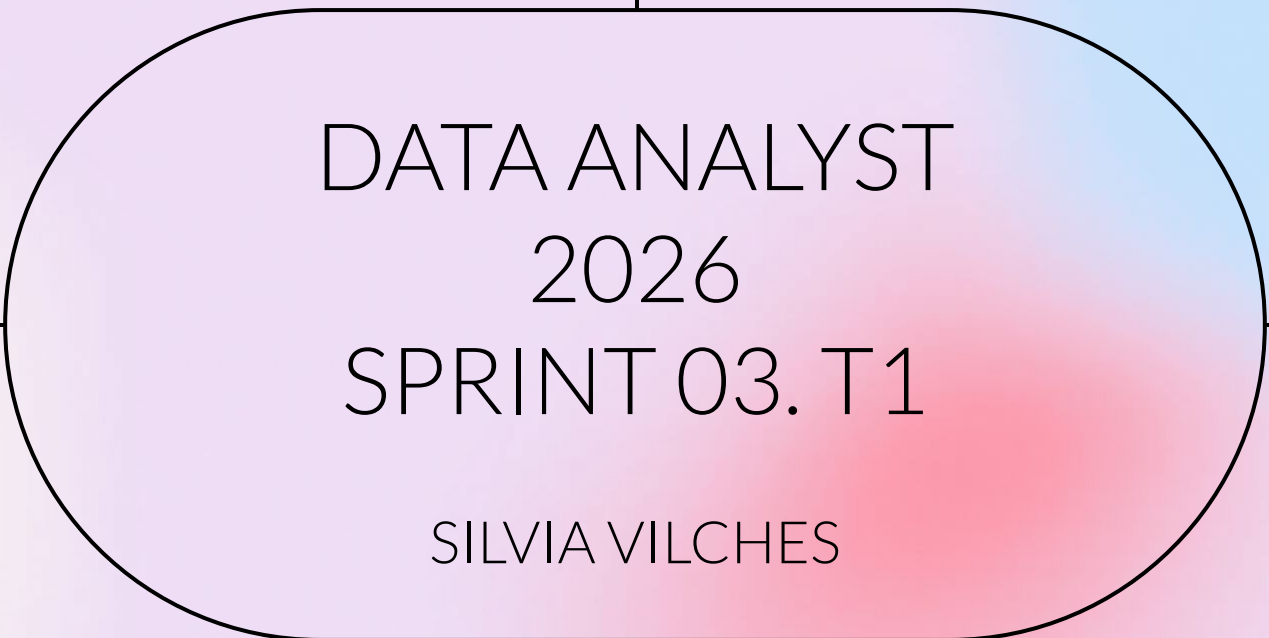




DATA ANALYST
2026
SPRINT 03.T1
SILVIA VILCHES

ÍNDICE



Pág. 03 - 15



Pág. 16 - 20



Pág. 21- XX



1

Ejercicio 1

Implementarás físicamente la arquitectura lógica creando tres conjuntos de datos dentro de un único proyecto para centralizar la facturación y los permisos.

👉 Crea un nuevo proyecto en Google Cloud con el nombre: `sprint3-analytics-[tu-nombre]`.

Creamos nuestro proyecto a través de la interfaz gráfica de GC. Añadimos el nombre que nos piden *sprint3-analytics-silvia*. No seleccionamos ninguna organización ya que no estamos abriendo este proyecto para ninguna empresa en particular.

Ya tenemos el proyecto creado.

Proyecto nuevo

 Tienes 9 projects restantes en tu cuota. Solicita un incremento o borra algunos proyectos. [Más información](#)

[Manage Quotas](#)

Nombre del proyecto *

sprint3-analytics-silvia

ID de proyecto *

sprint3analyticssilvia

El ID del proyecto puede contener letras minúsculas, números o guiones. Debe comenzar con una letra minúscula y terminar con una letra o un número.

Organización *

Sin organización

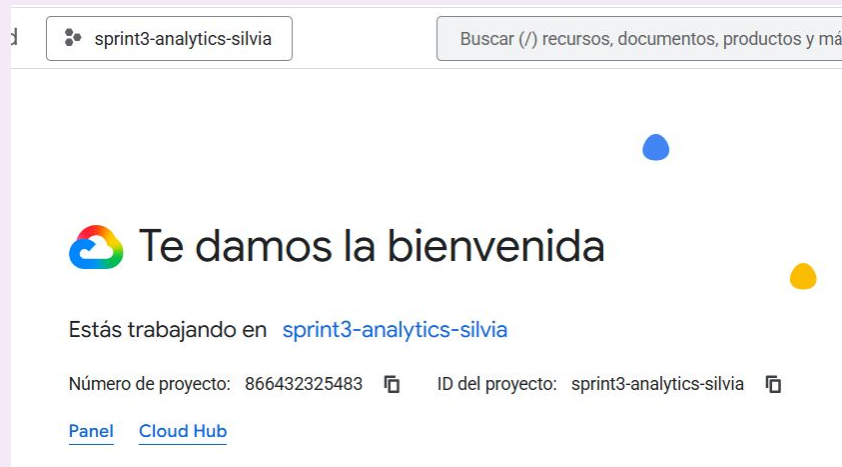
Selecciona una organización para vincularla a un proyecto. No podrás cambiar esta selección más adelante.

Recurso superior *

 Sin organización [Explorar](#)

Selecciona la organización o carpeta en la que se creará este recurso

[Crear](#) [Cancelar](#)



1

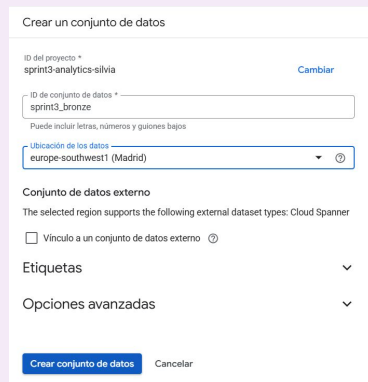
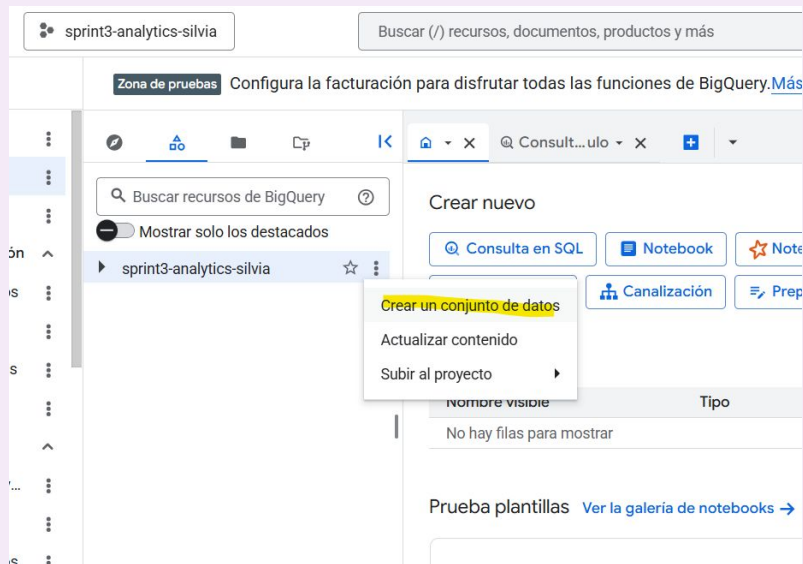
Ejercicio 1

Crea un conjunto de datos físico *sprint3_bronze* → Capa lógica Bronze (datos sin procesar).

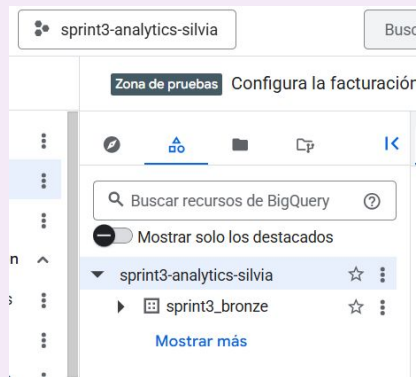
Función: Datos sin procesar tal y como llegan desde la fuente.

Método: Utilice la interfaz de usuario (UI) de BigQuery (apuntar y hacer clic).

Creemos nuestro primer dataset con la interfaz gráfica.



Le asignamos nuestro proyecto, el nombre *sprint3_bronze* que nos indican y la ubicación que ya hemos marcado anteriormente.



Tenemos el dataset creado en nuestro proyecto

1

Ejercicio 1

Crea un conjunto de datos físico *sprint3_silver* → Capa lógica Silver (datos limpios)
Función: Datos limpios, tipificados y de duplicados.
Método: Utilice código SQL (CREATE SCHEMA).

Creemos nuestro siguiente dataset con la una consulta de código SQL.

```
1 CREATE SCHEMA sprint3analyticssilvia.sprint3_silver
2 OPTIONS (location = 'europe-southwest1');
```

Nos indica que no se han
facturado datos para esta
consulta.

Resultados de la consulta	
Información del trabajo	Resultados
ID de trabajo	sprint3analyticssilvia:europe-southwest1.bqjob_55fcf6
Usuario	silviavilches.m@gmail.com
Ubicación	europe-southwest1
Hora de creación	14 may 2026, 12:03:22 p.m. UTC+2
Hora de inicio	14 may 2026, 12:03:22 p.m. UTC+2
Hora de finalización	14 may 2026, 12:03:24 p.m. UTC+2
Duración	1 s
Bytes procesados	0 B
Bytes facturados	0 B
Milisegundos de ranura	0
Prioridad del trabajo	INTERACTIVE

1

Ejercicio 1

Crea un conjuntos de datos físico *sprint3_gold* → Capa lógica Gold (datos empresariales)
 Función: Datos agregados listos para informes y paneles de control.
 Método: Utiliza Cloud Shell (línea de comandos bq).

Por último, creamos un dataset utilizando comandos *BQ* con Cloud Shell.

```

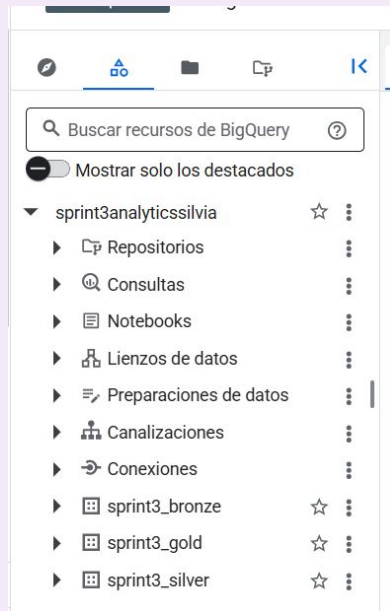
CLOUD SHELL
Terminal (sprint3analyticssilvia) x + v

Gemini CLI está disponible en la terminal de Cloud Shell. Pruébala escribiendo gemini. Más información

Welcome to Cloud Shell! Type "help" to get started, or type "gemini" to try prompting with Gemini CLI.
Your Cloud Platform project in this session is set to sprint3analyticssilvia.
Use 'gcloud config set project [PROJECT_ID]' to change to a different project.
silviavilches_m@cloudshell:~ (sprint3analyticssilvia)$ bq --location=europe-southwest1 \
--description="Capa lógica Gold (datos empresariales)" \
sprint3analyticssilvia:sprint3_gold
FATAL: Flags parsing error: Unknown command line flag 'description'
Run 'bq.py help' to get help
silviavilches_m@cloudshell:~ (sprint3analyticssilvia)$ bq location=europe-southwest1 \
sprint3analyticssilvia:sprint3_gold
FATAL: Command 'location=europe-southwest1' unknown
Run 'bq.py help' to get help
silviavilches_m@cloudshell:~ (sprint3analyticssilvia)$ bq location=EU \
sprint3analyticssilvia:sprint3_gold
FATAL: Command 'location=EU' unknown
Run 'bq.py help' to get help
silviavilches_m@cloudshell:~ (sprint3analyticssilvia)$ bq mk \
--dataset \
--location=europe-southwest1 \
sprint3analyticssilvia:sprint3_gold
Dataset 'sprint3analyticssilvia:sprint3_gold' successfully created.
silviavilches_m@cloudshell:~ (sprint3analyticssilvia)$
  
```

`bq mk \`
`--dataset \` – Indicamos qué queremos crear
`--location=europe-southwest1 \` – Misma localización del proyecto
`sprint3analyticssilvia:sprint3_gold` – Id del proyecto : nombre_dataset

Ya tenemos los 3 conjuntos de datos creados dentro de nuestro proyecto.



1

Ejercicio 2

Escribe y ejecuta las sentencias **CREATE EXTERNAL TABLE** para vincular los siguientes archivos al conjunto de datos *sprint3_bronze*. Presta especial atención a las «Notas técnicas», ya que no todos los archivos tienen el mismo formato.

Taula	D	Ruta al Núvol (URI)	Notes Tècniques
transaccio ns_raw	ERP	gs://bootcamp-data-analytics-public/ERP/transactions.csv	Delimitador ; (Punt i coma)

IMPORTANTE: Debemos especificar el delimitador punto y coma (;) que nos indica en las notas técnicas porque si lo omitimos, BigQuery interpreta todo el registro como un solo campo, no separa los valores.

```

8
9 CREATE EXTERNAL TABLE sprint3_bronze.transactions_raw
10 OPTIONS (
11   format = 'CSV',
12   uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv'],
13   skip_leading_rows = 1,
14   field_delimiter = ';'
15 );

```

✓ Esta secuencia de comandos procesará 0 B cuando se ejecute.

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

ⓘ Esta declaración creó una nueva tabla con el nombre transactions_raw.

`format = 'CSV'`, -- Indicamos el formato de nuestro archivo
`uris = ['gs://bootcamp-data-analytics-public/ERP/transactions.csv']`, --
 ubicación de nuestro archivo
`skip_leading_rows = 1`, -- no tener en cuenta la primera línea de registros
`field_delimiter = ';' -- delimitador ;`

Taula	D	Ruta al Núvol (URI)	Notes Tècniques
comp anies_ raw	ERP	gs://bootcamp-data-analytics-pub lic/ERP/companies.csv	Capçalera: Ignora la 1a fila. Defineix l'esquema manualment perquè tot és text.

IMPORTANTE: Debemos ignorar la primera fila. Todo es texto: debemos indicar cada campo y su tipo de dato manualmente.

```

20
21 CREATE EXTERNAL TABLE `sprint3_bronze.companies_raw` (
22   company_id STRING,
23   company_name STRING,
24   phone INT64,
25   email STRING,
26   Website STRING
27 )
28 OPTIONS (
29   format = 'CSV',
30   uris = ['gs://bootcamp-data-analytics-public/ERP/companies.csv'],
31   skip_leading_rows = 1
32 );

```

✓ Esta secuencia de comandos procesará 0 B cuando se ejecute.

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

ⓘ Esta declaración creó una nueva tabla con el nombre companies_raw.

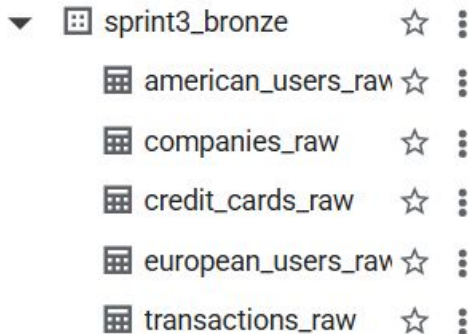
1

Ejercicio 2

Escribe y ejecuta las sentencias **CREATE EXTERNAL TABLE** para vincular los siguientes archivos al conjunto de datos *sprint3_bronze*. Presta especial atención a las «Notas técnicas», ya que no todos los archivos tienen el mismo formato.

Taula	D	Ruta al Núvol (URI)	Notes Tècniques
american_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/american_users.csv	Estàndard.
european_users_raw	CRM	gs://bootcamp-data-analytics-public/CRM/european_users.csv	Estàndard.
credit_cards_raw	CRM	gs://bootcamp-data-analytics-public/CRM/credit_cards.csv	Estàndard.

Tenemos las 5 tablas creadas.



El resto de tablas no tienen ninguna nota técnica específica. Ejecutamos.

```

35 # CREAT TABLA CREDIT_CARDS
36 CREATE EXTERNAL TABLE sprint3_bronze.credit_cards_raw
37 OPTIONS (
38   format = 'CSV',
39   uris = ['gs://bootcamp-data-analytics-public/CRM/credit_cards.csv'],
40   skip_leading_rows = 1
41 );
42
43
44 # CREAT TABLA AMERICAN_USERS
45 CREATE EXTERNAL TABLE sprint3_bronze.american_users_raw
46 OPTIONS (
47   format = 'CSV',
48   uris = ['gs://bootcamp-data-analytics-public/CRM/american_users.csv'],
49   skip_leading_rows = 1
50 );
51
52
53 # CREAT TABLA EUROPEAN_USERS
54 CREATE EXTERNAL TABLE sprint3_bronze.european_users_raw
55 OPTIONS (
56   format = 'CSV',
57   uris = ['gs://bootcamp-data-analytics-public/CRM/european_users.csv'],
58   skip_leading_rows = 1
59 );

```

1

Ejercicio 3

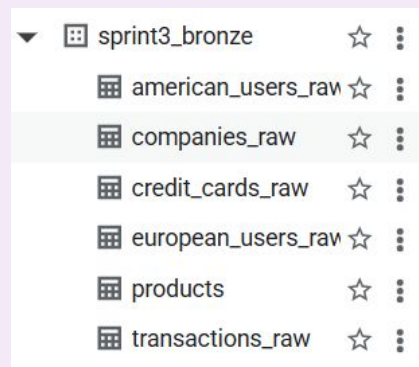
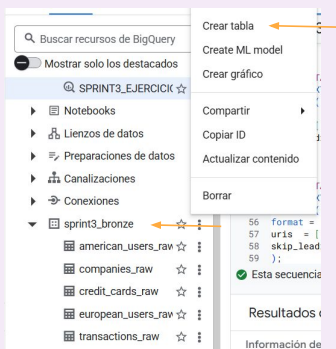
Falta el catálogo de productos. Esta información no se encuentra en el Data Lake; utiliza el archivo `products.csv` del *sprint 2*.

- » Sube manualmente este archivo al conjunto de datos *sprint3_bronze* y crea la tabla *products_raw*.
- » Ten en cuenta que, por el momento, esta será la única tabla nativa.

Desde nuestro dataset le damos a **Crear Tabla**.

Identificamos la fuente, el archivo y el tipo de archivo y el destino, es decir, donde se alojará esta tabla.

Que esta tabla nativa sea la única nativa implica que los datos se copian en BigQuery y las consultas serán más rápidas, ya que los datos se almacenan internamente a diferencia del resto de tablas.

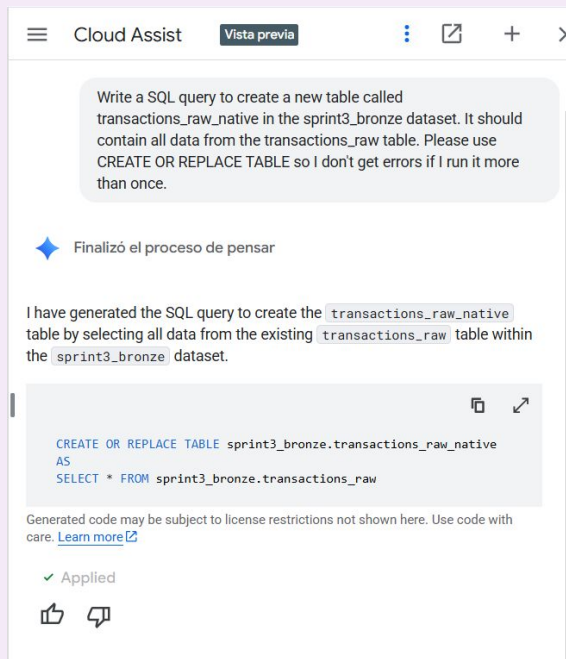


1

Ejercicio 4

Crea una nueva tabla llamada *sprint3_bronze.transactions_raw_native* que sea una copia exacta de tu tabla externa *transactions_raw*. Requisito de «programación en pareja con IA»: No escribas el código SQL manualmente. Utiliza el asistente de IA integrado (Gemini) de BigQuery (o el chat lateral) para generar la instrucción.

Activar Gemini y añadir el prompt que nos da la actividad.



Cloud Assist Vista previa

Write a SQL query to create a new table called `transactions_raw_native` in the `sprint3_bronze` dataset. It should contain all data from the `transactions_raw` table. Please use `CREATE OR REPLACE TABLE` so I don't get errors if I run it more than once.

Finalizó el proceso de pensar

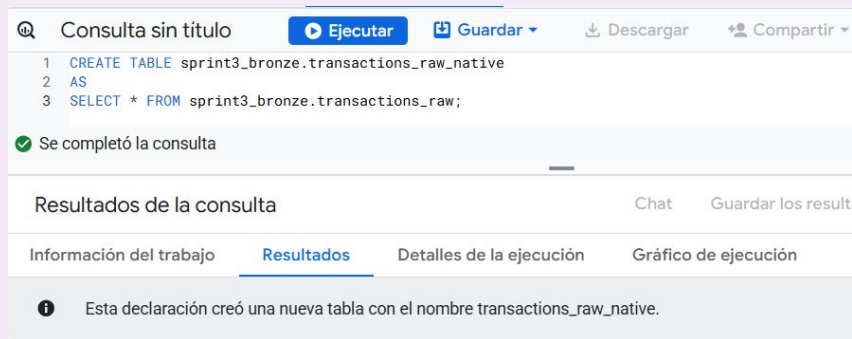
I have generated the SQL query to create the `transactions_raw_native` table by selecting all data from the existing `transactions_raw` table within the `sprint3_bronze` dataset.

```
CREATE OR REPLACE TABLE sprint3_bronze.transactions_raw_native
AS
SELECT * FROM sprint3_bronze.transactions_raw
```

Generated code may be subject to license restrictions not shown here. Use code with care. [Learn more](#)

✓ Applied

El código dado es correcto, solo faltaba cerrarlo ;



Consulta sin título Ejecutar Guardar Descargar Compartir

```
1 CREATE TABLE sprint3_bronze.transactions_raw_native
2 AS
3 SELECT * FROM sprint3_bronze.transactions_raw;
```

✓ Se completó la consulta

Resultados de la consulta Chat Guardar los resultados

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

Esta declaración creó una nueva tabla con el nombre `transactions_raw_native`.

Ejercicio 4

Compara el coste de leer una sola columna (por ejemplo, «id») de la tabla externa frente a la tabla nativa. Utiliza los parámetros *bytes procesados* y *bytes facturados* para realizar la comparación.

» ¿Cuánto «pesa» la consulta en «transactions_raw»?

» ¿Cuánto «pesa» la misma consulta en «transactions_raw_native»?

CONSULTA TABLA EXTERNA

```
1 SELECT id
2 FROM `sprint3_bronze.transactions_raw`;
3
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Visualización

ID de trabajo	sprint3analyticssilvia:europa-southwest1
Usuario	silviavilches.m@gmail.com
Ubicación	europa-southwest1
Hora de creación	14 may 2026, 9:06:45 p.m. UTC+2
Hora de inicio	14 may 2026, 9:06:45 p.m. UTC+2
Hora de finalización	14 may 2026, 9:06:47 p.m. UTC+2
Duración	1 s
Bytes procesados	12.61 MB
Bytes facturados	13 MB

CONSULTA TABLA NATIVA

```
1 SELECT id
2 FROM `sprint3_bronze.transactions_raw_native`;
3
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Visualización JS

ID de trabajo	sprint3analyticssilvia:europa-southwest1.bqu:
Usuario	silviavilches.m@gmail.com
Ubicación	europa-southwest1
Hora de creación	14 may 2026, 9:08:22 p.m. UTC+2
Hora de inicio	14 may 2026, 9:08:22 p.m. UTC+2
Hora de finalización	14 may 2026, 9:08:23 p.m. UTC+2
Duración	0 s
Bytes procesados	3.62 MB
Bytes facturados	10 MB

La consulta en la tabla externa supone un **30% más de bytes facturados** y prácticamente cuadruplica los bytes procesados lo que supone un incremento en el gasto de la empresa que a largo plazo será importante. Esto ocurre porque BQ debe leer toda la tabla para encontrar la columna ID mientras que en la tabla nativa, que ya tiene almacenada, ya sabe donde está. Podemos verlo también en la duración de la consulta, al tener que buscarlo, tarda más.

Ejercicio 4

Ejecuta la consulta **`SELECT * FROM transactions_raw_native LIMIT 10`** en la tabla externa.

✓ Comprueba el «dry run». ¿Ha bajado el coste respecto a no añadir un **LIMIT**?

Explica brevemente por qué esto es una trampa para principiantes.

CONSULTA SIN LIMIT 10

```
1 SELECT *
2 FROM `sprint3_bronze.transactions_raw_native`;
3
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

ID de trabajo	sprint3analyticssilvia:europa-southwest1
Usuario	silviavilches.m@gmail.com
Ubicación	europa-southwest1
Hora de creación	14 may 2026, 9:11:22 p.m. UTC+2
Hora de inicio	14 may 2026, 9:11:22 p.m. UTC+2
Hora de finalización	14 may 2026, 9:11:23 p.m. UTC+2
Duración	1 s
Bytes procesados	10.87 MB
Bytes facturados	11 MB

CONSULTA CON LIMIT 10

```
1 SELECT *
2 FROM `sprint3_bronze.transactions_raw_native`
3 LIMIT 10;
4
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo

Resultados

Visualización

JSON

ID de trabajo	sprint3analyticssilvia:europa-southwest1.bq
Usuario	silviavilches.m@gmail.com
Ubicación	europa-southwest1
Hora de creación	14 may 2026, 9:13:37 p.m. UTC+2
Hora de inicio	14 may 2026, 9:13:37 p.m. UTC+2
Hora de finalización	14 may 2026, 9:13:37 p.m. UTC+2
Duración	0 s
Bytes procesados	10.87 MB
Bytes facturados	11 MB
Milisegundos de	28

No, el **coste NO ha bajado**. Ambos consumen exactamente 10,87 MB, tanto con **LIMIT 10** como sin él.

Un principiante puede pensar que BQ solo lee las 10 primeras líneas pero BQ escanea primero todas las columnas de **`SELECT *`** y después aplica el **LIMIT**. El **LIMIT** se aplica en el paso final, una vez que los datos han sido leídos y facturados.

1

Ejercicio 5

Tu jefe quiere saber cuáles fueron los 5 días con mayor volumen de ingresos de 2021.

🔥 Reto: Es probable que el campo **TIMESTAMP** sea un **STRING**. Tendrás que consultar las funciones de BigQuery (**SUBSTR**, **CAST**, **PARSE_TIMESTAMP**) para filtrar el año y agrupar por fecha correctamente.

Efectivamente, mi primera consulta me daba error porque la fecha estaba en formato **STRING**. Debo convertir esta columna en una fecha para poder extraer tanto la fecha como el año para poder filtrar.

```
1 SELECT
2   DATE(CAST(timestamp AS TIMESTAMP)) AS date,
3   ROUND(SUM(amount),2) AS total_amount
4 FROM `sprint3_bronze.transactions_raw`
5 WHERE EXTRACT(YEAR FROM CAST(timestamp AS TIMESTAMP)) = 2021
6 AND declined = 0
7 GROUP BY date
8 ORDER BY total_amount DESC
9 LIMIT 5;
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo		Resultados	Visualización	JSON
Fila	date ▾	total_amount ▾		
1	2021-10-27	11440.81		
2	2021-01-22	11328.84		
3	2021-02-11	11182.16		
4	2021-04-29	11119.01		
5	2021-03-27	10898.51		

· **DATE(CAST(timestamp AS TIMESTAMP)) AS date**

Convertimos el campo *timestamp* en tipo fecha **TIMESTAMP** con la cláusula **CAST**. De aquí, extraemos solo la fecha **DATE** porque esta columna también contiene la hora de la transacción. Llamamos a la columna “*date*”.

· **ROUND(SUM(amount),2) AS total_amount**

Redondeamos el resultado de la suma de las transacciones por día a solo 2 decimales, aparecen muchos en la primera consulta.

· **WHERE EXTRACT(YEAR FROM CAST(timestamp AS TIMESTAMP)) = 2021**

Para filtrar por el año 2021, de nuevo convertimos el campo *timestamp* en tipo fecha **TIMESTAMP** con la cláusula **CAST** y seleccionamos solo el año, información necesaria para el resultado que nos piden.

1

Ejercicio 6

Necesitamos un informe que cruce datos.

🌿 **Tarea:** Enumera el nombre, el país y la fecha de las transacciones realizadas por empresas que hayan llevado a cabo transacciones de entre 100 € y 200 € en una de estas fechas: 29-04-2015, 20-07-2018 o 13-03-2024.

He creado una consulta combinando las tablas de *companies* y *transactions*.

He tenido que renombrar las tablas porque no me reconocía la tabla *companies*.

```

1 SELECT t2.company_name, t2.country, date(t1.timestamp) AS date
2 FROM
3 `sprint3_bronze.transactions_raw_native` AS t1
4 INNER JOIN
5 `sprint3_bronze.companies_raw` AS t2
6 ON t2.company_id = t1.business_id
7 WHERE t1.amount BETWEEN 100 AND 200
8 AND date(t1.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13');

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guarda](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	company_name	country	date		
1	Fusce Corp.	United States	2018-07-20		
2	Auctor Mauris Vel LLP	United States	2015-04-29		
3	Non Magna LLC	United Kingdom	2015-04-29		
4	Donec Fringilla PC	France	2015-04-29		
5	Magna Incorporated	New Zealand	2018-07-20		
6	Ac Ullamcorper	United Kingdom	2024-03-13		

Resultados por página: 50 ▼

IMPORTANTE: Habría sido más rápido utilizar la cláusula **USING** pero en este caso los campos no se llamaban igual, en la tabla *transaction* el campo con el ID de la compañía se llama *business_id*.

La consulta podría haber sido así:

```

SELECT t2.company_name, t2.country, date(t1.timestamp) AS date
FROM
`sprint3_bronze.transactions_raw_native` AS t1
INNER JOIN
`sprint3_bronze.companies_raw` AS t2
USING(company_id)
WHERE t1.amount BETWEEN 100 AND 200
AND date(t1.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13');

```



2

Ejercicio 1

Crearemos la capa depurada («Silver») para los productos. Tienes la tabla «*sprint3_bronze.products_raw*» cargada desde el CSV. El equipo de Data Governance te pide crear una tabla de productos depurada en «*sprint3_silver.products_clean*» que cumpla estas normas de calidad:

- » **Estandarización de nombres:** la columna id original es ambigua; cámbiale el nombre a `product_id`. Simplifica la columna `product_name` a `name`.
- » **Limpieza de ID:** el campo `warehouse_id` tiene el formato antiguo «WH-4». Elimina el prefijo «WH-» y convierte el valor a un entero (INT64).
- » **Garantía de precios:** Asegúrate de que el precio sea un número (FLOAT64), sin símbolos de moneda.
- » **Otras columnas:** Mantén el campo «weight» tal y como está.

ar codigo que tenga en cuenta otros formatos

CONSULTA / CÓDIGO:

· `CREATE OR REPLACE TABLE `sprint3_silver.products_clean` AS`

Creamos la tabla a partir de un **SELECT**, a partir de otra tabla

· `id AS product_id // product_name AS name`

Cambiamos los nombres de los campos con **AS**

· `CAST(price AS FLOAT64) AS price`

Cambiamos el tipo del registro, con la cláusula **CAST** a tipo *numérico decimal* con **FLOAT64**

· `CAST(REPLACE(warehouse_id, 'WH-', '' ) AS INT64) AS warehouse_id`

Cambiamos el tipo del registro, con la cláusula **CAST** a tipo *numérico entero* con **INT64**. Con **REPLACE** vamos a sustituir todos los “WH-” de los registros por nada “”.

· `FROM `sprint3_bronze.products``

El origen, la tabla que queremos duplicar

▼	🏠 sprint3_silver	☆	⋮
	🏠 products_clean	☆	⋮

```

4 CREATE OR REPLACE TABLE `sprint3_silver.products_clean` AS
5 SELECT
6   id AS product_id,
7   product_name AS name,
8   CAST(price AS FLOAT64) AS price,
9   weight,
10  colour,
11  CAST(REPLACE(warehouse_id, 'WH-', '' ) AS INT64) AS warehouse_id,
12  FROM `sprint3_bronze.products`;

```

✓ Esta consulta procesará 5.62 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfi

❗ Esta declaración creó una nueva tabla con el nombre `products_clean`.

2

Ejercicio 2

Crearemos la tabla definitiva ``sprint3_silver.transactions_clean``, que garantiza lo siguiente:

- » **Estandarización de nombres:** la columna id original es ambigua; cámbiale el nombre a `transaction_id`.
- » **Robustez del importe:** utiliza `SAFE_CAST` en el campo del importe. Si falla, sustitúyelo por 0 (utilizando `IFNULL`).
- » **Fechas reales:** convierte el campo de marca de tiempo (que es `STRING`) a un tipo `TIMESTAMP` real.
- » **Coordenadas:** Asegúrate de que `lat` y `longitude` sean `FLOAT64` (utiliza `SAFE_CAST` por seguridad).
- » **Resto de campos:** Déjalos tal y como están.

CONSULTA / CÓDIGO:

· `CREATE OR REPLACE TABLE `sprint3_silver.transactions_clean` AS`

Creamos la tabla a partir de un `SELECT`, a partir de otra tabla

· `id AS transaction_id`

Cambiamos los nombres de los campos con `AS` para mayor claridad

· `IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount`

Cambiamos el tipo del registro, con la cláusula `SAFE_CAST` a tipo *numérico decimal* con `FLOAT64`. A diferencia de `CAST`, `SAFE_CAST` nos devuelve cualquier error como `NULL`. Pero al implementar la cláusula `IFNULL(0)`, cualquier `NULL` nos lo transforma en "0"

· `CAST(timestamp AS TIMESTAMP) AS timestamp`

Cambiamos el tipo del registro, con la cláusula `CAST` a tipo *fecha* con `TIMESTAMP`.

· `SAFE_CAST(lat AS FLOAT64) AS lat // SAFE_CAST(longitude AS FLOAT64) AS longitude`

Cambiamos el tipo del registro, con la cláusula `SAFE_CAST` a tipo *número decimal* con `FLOAT64`. Cualquier error, nos devuelve `NULL`.

```
1 CREATE OR REPLACE TABLE `sprint3_silver.transactions_clean` AS
2 SELECT
3   id AS transaction_id,
4   card_id,
5   business_id,
6   CAST(timestamp AS TIMESTAMP) AS timestamp,
7   IFNULL(SAFE_CAST(amount AS FLOAT64), 0) AS amount,
8   declined,
9   product_ids,
10  user_id,
11  SAFE_CAST(lat AS FLOAT64) AS lat,
12  SAFE_CAST(longitude AS FLOAT64) AS longitude,
13  FROM `sprint3_bronze.transactions_raw`;
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo

Resultados

Detalles de la ejecución



Esta declaración reemplazó la tabla denominada transactions_clean.

- ▼ sprint3_silver
 - products_clean ☆ ⋮
 - transactions_clean ☆ ⋮

2

Ejercicio 3

Crea la tabla `sprint3_silver.users_combined`. Utiliza **UNION ALL** para unir los usuarios de EE. UU. y Europa en una única lista maestra. Añade una columna calculada «*origin*» para indicar de dónde proceden.

» **Estandarización de nombres:** La columna id original es ambigua; cámbiale el nombre a `user_id`.

```
1 CREATE OR REPLACE TABLE sprint3_silver.users_combined AS
2 SELECT
3   id AS user_id,
4   name,
5   surname,
6   phone,
7   email,
8   birth_date,
9   country,
10  city,
11  postal_code,
12  address,
13  'USA' AS origin
14 FROM sprint3_bronze.american_users_raw
15
16 UNION ALL
17
18 SELECT
19   id AS user_id,
20   name,
21   surname,
22   phone,
23   email,
24   birth_date,
25   country,
26   city,
27   postal_code,
28   address,
29   'EU' AS origin
30 FROM sprint3_bronze.european_users_raw;
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución

❗ Esta declaración creó una nueva tabla con el nombre `users_combined`.

sprint3_silver	☆	⋮
products_clean	☆	⋮
transactions_clean	☆	⋮
users_combined	☆	⋮

CONSULTA / CÓDIGO:

• **CREATE OR REPLACE TABLE** `sprint3\_silver.users\_combined` **AS**

Creamos la tabla a partir de un **SELECT**, a partir de otra tabla

• `id` **AS** `user_id`

Cambiamos los nombres de los campos con **AS** para mayor claridad

• El resto de campos los dejamos igual

• **UNION ALL**

Combinamos/unimos con **UNION ALL** ambas tablas de usuarios que contienen la misma información y estructura pero diferentes ubicaciones.

• `'EU'` **AS** `origin` // `'USA'` **AS** `origin`

Añadimos una nueva columna con la ubicación de origen de cada usuario. En este caso, tenemos claro que son de EEUU o Europa así que añadimos y nombramos la columna con **AS**.

Ejercicio 3

Crea la tabla `sprint3_silver.users_combined`. Utiliza **UNION ALL** para unir los usuarios de EE. UU. y Europa en una única lista maestra. Añade una columna calculada «*origin*» para indicar de dónde proceden.

» **Estandarización de nombres:** La columna id original es ambigua; cámbiale el nombre a `user_id`.

Comprobamos que la columna 'origin' se ha añadido correctamente. Filtraremos por un solo país para evitar que lea toda la tabla y generar más costes.

Filtro : Canadá

```
6
7 SELECT country, origin,
8 FROM
9 `sprint3analyticssilvia.sprint3_silver.users_combined`
10 WHERE country = 'Canada';
```

✓ Esta consulta procesará 69.51 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JSC
Fila	country	origin	
1	Canada	USA	
2	Canada	USA	
3	Canada	USA	

Filtro : Alemania

```
6
7 SELECT country, origin,
8 FROM
9 `sprint3analyticssilvia.sprint3_silver.users_combined`
10 WHERE country = 'Germany';
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo	Resultados	Visualización	JS
Fila	country	origin	
1	Germany	EU	
2	Germany	EU	
3	Germany	EU	

2

Ejercicio 4

- » 1. Crea la tabla `sprint3_silver.companies_clean`. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery.
 - » 2. Crea la tabla `sprint3_silver.credit_cards_clean`. Copia los datos tal cual, pero asegúrate de que sea una tabla nativa de BigQuery.
- Si es necesario, cambia el nombre del id según corresponda.

Creemos una tabla nativa, utilizando **CREATE TABLE** (no **EXTERNAL**).
En este caso, he renombrado el campo `company_id` para que coincida con el campo en la tabla de `'transactions_clean'`.

```

4 CREATE OR REPLACE TABLE `sprint3_silver.companies_clean` AS
5 SELECT
6   company_id AS business_id,
7   company_name,
8   phone,
9   email,
10  country,
11  Website,
12  FROM `sprint3_bronze.companies_raw`;

```

Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución

Esta declaración creó una nueva tabla con el nombre `companies_clean`.

En la tabla de `credit_cards_id`, he renombrado el campo `id` para que coincida con el campo en la tabla de `'transactions_clean'`.

```

4 CREATE OR REPLACE TABLE `sprint3_silver.credits_cards_clean` AS
5 SELECT
6   id AS card_id,
7   user_id,
8   iban,
9   pan,
10  pin,
11  cvv,
12  track1,
13  track2,
14  expiring_date,
15  FROM `sprint3_bronze.credit_cards_raw`;

```

Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de

Esta declaración creó una nueva tabla con el nombre `credits_cards_clean`.

Todas las tablas están creadas.

sprint3_silver	☆	⋮
companies_clean	☆	⋮
credits_cards_clean	☆	⋮
products_clean	☆	⋮
transactions_clean	☆	⋮
users_combined	☆	⋮



3

Ejercicio 1

Crea una vista llamada *sprint3_gold.v_marketing_kpis* que muestre la siguiente información para cada empresa:

- » Nombre de la empresa, teléfono y país (de: *companies_clean*).
- » Compra media (**AVG**(amount) de *transactions_clean*).
- » Clasificación de cliente (Lógica):
 - Crea una columna calculada llamada *client_tier*.
 - Si la compra media es superior a 260 €, etiquétala como «Premium».
 - Si es igual o inferior, etiquétala como «Standard».

Creamos una vista, utilizando **CREATE VIEW**.

- **SELECT** – seleccionamos los campos que nos piden (name, phone y country)
- **AVG** - calculamos la media del campo “amount” y le damos nombre a esta columna.
- **CASE** – definimos una condición. En este caso solo es necesario especificar “Premium”; si no la cumple será “Standard”. Le damos nombre “client_tier” a esta columna nueva calculada.
- **INNER JOIN** – combinamos las tablas *companies_clean* y *transactions_clean*, de donde procede nuestra información.
- **USING** – En lugar de utilizar **ON**, **USING** es más efectivo ya que ambas columnas “business_id” coinciden en nombre.
- **GROUP BY** – agrupamos por los campos que aparecen en el **SELECT**.

▼ sprint3_gold ☆ ⋮
 v_marketing_kpis ☆ ⋮

Tenemos la vista creada.

```

9 CREATE VIEW `sprint3analyticssilvia.sprint3_gold.v_marketing_kpis`
10 AS
11 SELECT
12   companies.company_name,
13   companies.phone,
14   companies.country,
15   AVG(transactions.amount) AS avg_purchase,
16   CASE
17     WHEN AVG(transactions.amount) > 260 THEN "Premium"
18     ELSE "Standard"
19   END
20   AS client_tier
21 FROM `sprint3analyticssilvia.sprint3_silver.companies_clean` AS companies
22 INNER JOIN
23   `sprint3analyticssilvia.sprint3_silver.transactions_clean` AS transactions
24   USING(business_id)
25 GROUP BY companies.company_name, companies.phone, companies.country;
```

✓ Se completó la consulta

Resultados de la consulta

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejec

❗ Esta declaración creó una nueva vista llamada v_marketing_kpis.

3

Ejercicio 1

Entrega:

Ejecuta una consulta SELECT* en tu nueva vista, ordenando los resultados de modo que los clientes «Premium» aparezcan primero, y luego aquellos con la compra media más alta.

```
4  
5 SELECT *  
6 FROM `sprint3analyticssilvia.sprint3_gold.v_marketing_kpis`  
7 ORDER BY client_tier ASC, avg_purchase DESC;
```

✓ Esta consulta procesará 1.53 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

[Chat](#) [Guardar los resultados ▼](#)

Información del trabajo						Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	company_name ▼	phone ▼	country ▼	avg_purchase ▼	client_tier ▼					
1	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.8671595168...	Premium					
2	Pretium Neque Corp.	07 77 48 55 28	Australia	276.1583295711...	Premium					
3	Urna Convallis Associates	06 01 24 77 04	United States	274.2350109409...	Premium					
4	At Associates	09 56 61 10 65	New Zealand	272.2148701298...	Premium					
5	Metus Vitae Associates	08 25 44 40 66	Australia	270.0809647058...	Premium					
6	Aliquet Diam Limited	02 76 61 47 46	United States	269.5991810344...	Premium					
7	Nec Luctus LLC	02 14 71 75 73	Norway	268.6048370927...	Premium					
8	Neque Tellus Incorporated	04 43 18 34 19	Ireland	267.8503720930...	Premium					

3

Ejercicio 2

Crea una tabla `sprint3_gold.product_sales_ranking` que contenga el inventario completo de productos y cuántas veces se ha vendido cada uno.

- » **Detalle del producto:** Debe incluir `product_id`, `name`, `price` y `color` (`products_clean`)
- » **Métrica de negocio:** una columna nueva `total_sold` que cuente cuántas veces aparece este producto en las transacciones.
- » **Integridad:** Deben aparecer todos los productos, incluso los que tienen 0 ventas.

SUBCONSULTA:

Primero necesito convertir a **INT64** el campo "`product_ids`" de la tabla `transactions_clean` que es **STRING**.

Con **UNNEST** convertimos el campo "`product_ids`" que contiene muchos valores en muchas filas individuales, una por cada id.

Necesito hacer un **SPLIT** para separar cada elemento de ese registro e indicar que están separados por una coma y un espacio.

```
7 SELECT CAST(product_id AS INT64) AS product_id
8 FROM `sprint3_silver.transactions_clean`,
9 UNNEST(SPLIT(product_ids, ',')) AS product_id;
```

✓ Esta consulta procesará 970.03 KB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Información del trabajo Resultados Visualización

Fila	product_id	
46	3	
47	81	
48	94	
49	95	
50	66	

CONSULTA FINAL:

Seleccionamos los campos que nos piden y combinamos la tabla `products_clean` con la subconsulta anterior que he llamado `product_table`.

```
3 CREATE TABLE sprint3_gold.product_sales_ranking AS
4 SELECT products_clean.product_id, products_clean.name, products_clean.price, products_clean.colour,
5 COUNT(product_table.product_id) AS total_sold
6 FROM `sprint3_silver.products_clean` AS products_clean
7 LEFT JOIN (SELECT CAST(product_id AS INT64) AS product_id
8            FROM `sprint3_silver.transactions_clean`,
9            UNNEST(SPLIT(product_ids, ',')) AS product_id) AS product_table
10 ON products_clean.product_id = product_table.product_id
11 GROUP BY products_clean.product_id, products_clean.name, products_clean.price, products_clean.colour;
```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar

Información del trabajo Resultados Detalles de la ejecución Gráfico de ejecución

1 Esta declaración creó una nueva tabla con el nombre `product_sales_ranking`.

Tabla creada..

sprint3_gold	☆	⋮
product_sales_ranki	☆	⋮
v_marketing_kpis	☆	⋮

Ejercicio 3

Exporta los datos de la tabla «product_sales_ranking» a Google Sheets o descarga el archivo CSV en tu ordenador.

👉 Entrega:

Una captura de pantalla que muestre el archivo (Excel/Hojas de cálculo) abierto con los datos correctamente formateados.

GOOGLE SHEETS

docs.google.com/spreadsheets/d/1aTS_HLK_Q_SoChQ3H8

product_sales_ranking

Archivo Editar Ver Insertar Formato Datos Herramientas Extensiones

Menús

100%

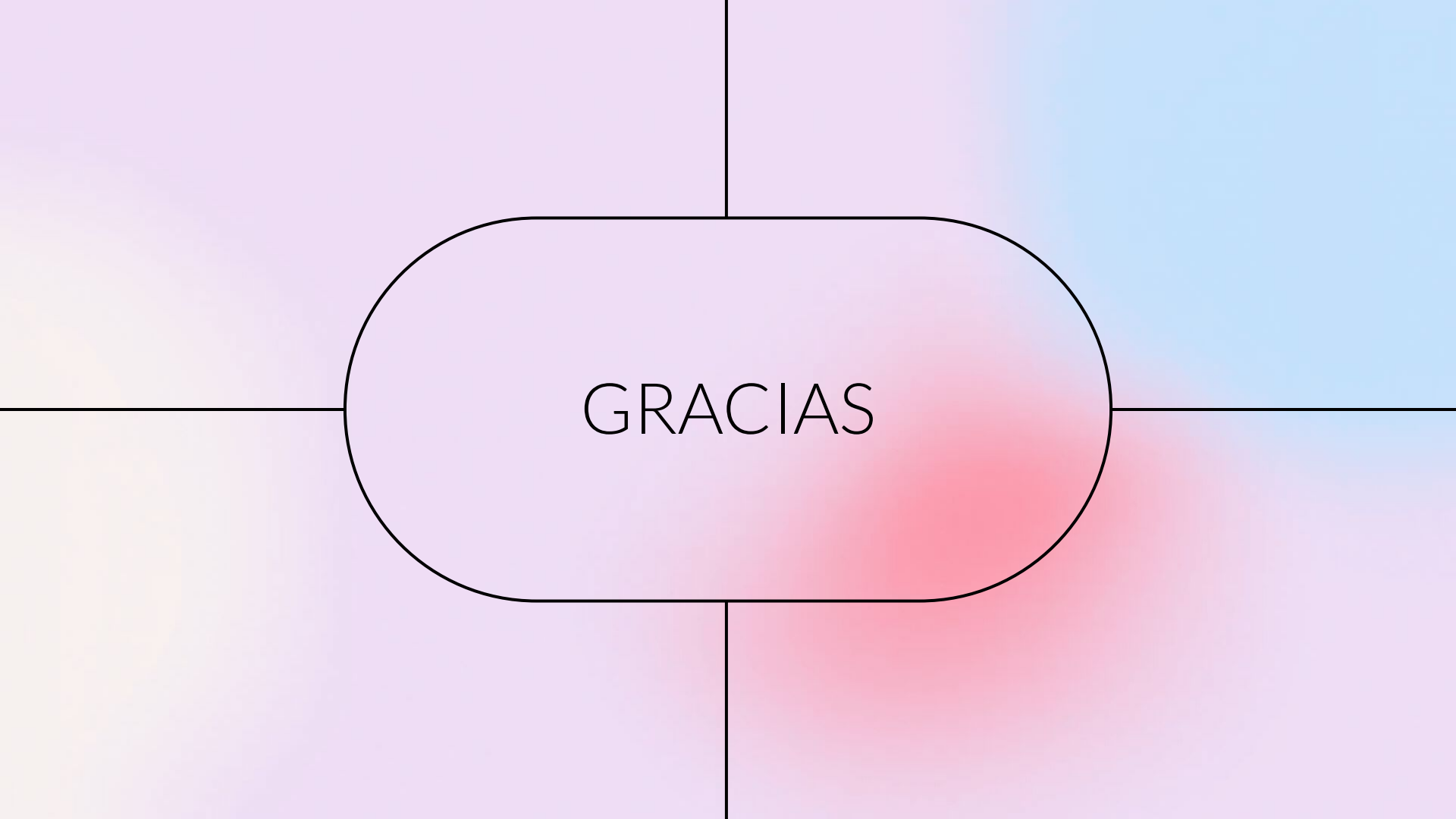
€ % 0.00 123 Predet...

	A	B	C	D	E
1	product_id	name	price	colour	total_sold
2	80	Tarly Stark	14.1	#020202	2496
3	40	tatooine dooku s	73.31	#070707	2488
4	32	north	178.28	#0f0f0f	2561
5	29	Tully maester Tarly	167.2	#111111	2635
6	4	warden south du	71.89	#111111	2584
7	11	Karstark Dorne	49.7	#141414	2573
8	86	Stark kingsblood	189.21	#141414	2528
9	33	duel warden	127.09	#191919	2597
10	67	Winterfell	195.94	#1c1c1c	2552
11	99	the duel	151.78	#212121	2514
12	51	north Tully	30.14	#262626	2466
13	96	dooku solo	20.92	#282828	2506
14	58	solo palpatine ch	130.59	#2b2b2b	2567
15	46	giantsblood mae	58.78	#2b2b2b	2513
16	13	palpatine chewbi	139.59	#2b2b2b	2560
17	25	skywalker ewok	157.53	#2b2b2b	2521
18	68	Stark Karstark	167.37	#333333	2589
19	84	Karstark Tully D	92.91	#353535	2545
20	8	Winterfell	32.37	#383838	2503
21	53	kingsblood Littlef	137.81	#3a3a3a	2564
22	94	in south Winterfe	51.94	#3a3a3a	2479
23	19	dooku solo	60.33	#3f3f3f	2449
24	31	Lannister	85.02	#3f3f3f	2580
25	71	Tully Dorne	103.73	#424242	2462
26	72	north	161.6	#474747	2573
27	85	bastard of north	62.62	#474747	2539
28	42	sith	83.51	#494949	2425

.CSV

G24

	A	B	C	D	E	F
1	product_id	name	price	colour	total_sold	
2	80	Tarly Stark	14.1	#020202	2496	
3	40	tatooine dooku solo	73.31	#070707	2488	
4	32	north	178.28	#0f0f0f	2561	
5	29	Tully maester Tarly	167.2	#111111	2635	
6	4	warden south duel	71.89	#111111	2584	
7	11	Karstark Dorne	49.7	#141414	2573	
8	86	Stark kingsblood	189.21	#141414	2528	
9	33	duel warden	127.09	#191919	2597	
10	67	Winterfell	195.94	#1c1c1c	2552	
11	99	the duel	151.78	#212121	2514	
12	51	north Tully	30.14	#262626	2466	
13	96	dooku solo	20.92	#282828	2506	
14	58	solo palpatine chewbacca	130.59	#2b2b2b	2567	
15	46	giantsblood maester Karstark	58.78	#2b2b2b	2513	
16	13	palpatine chewbacca	139.59	#2b2b2b	2560	
17	25	skywalker ewok	157.53	#2b2b2b	2521	
18	68	Stark Karstark	167.37	#333333	2589	
19	84	Karstark Tully Dorne	92.91	#353535	2545	
20	8	Winterfell	32.37	#383838	2503	
21	53	kingsblood Littlefinger the	137.81	#3a3a3a	2564	
22	94	in south Winterfell	51.94	#3a3a3a	2479	
23	19	dooku solo	60.33	#3f3f3f	2449	
24	31	Lannister	85.02	#3f3f3f	2580	
25	71	Tully Dorne	103.73	#424242	2462	
26	72	north	161.6	#474747	2573	
27	85	bastard of north	62.62	#474747	2539	
28	42	sith	83.51	#494949	2425	
29	88	Stannis warden south	167.15	#515151	2587	
30	23	riverlands north	169.96	#545454	2593	
31	82	kingsblood Littlefinger the	12.45	#545454	2560	
32	66	mustafar jinn	2.12	#545454	2601	
33	43	duel	59.8	#5b5b5b	2414	
34	56	Direwolf riverlands	167.07	#5b5b5b	2537	
35	30	Karstark warden	79.53	#606060	2468	
36	89	skywalker ewok	172.78	#636363	2538	



GRACIAS